Comparing DQN Variants and PPO on Pong and LunarLander

NYCU Artificial Intelligence – Project #3

Student ID: 313553058

May 2026

1 Research Question and Motivation

Reinforcement learning (RL) algorithms differ in fundamental ways – some learn the action-value function directly (e.g., DQN), others learn a policy through gradient ascent on expected return (e.g., PPO). Each design choice comes with assumptions about the task: action space type, on/off-policy sample reuse, exploration mechanism, and stability under function approximation. This project asks three concrete questions.

RQ1. On the same Atari task (Pong), how do three off-policy value-based methods that share the same code path – *Vanilla DQN* [Mnih et al., 2015], *Double DQN* [van Hasselt et al., 2016], and *Dueling Double DQN* [Wang et al., 2016] – differ in learning speed, final return, and Q-value calibration?

RQ2. On a different domain with low-dimensional vector observations (LunarLander), is an off-policy value-based method (DQN) or an on-policy policy-gradient method (PPO, Schulman et al., 2017) preferable? We compare wall-clock cost, sample efficiency, and final score.

RQ3. How sensitive are the resulting agents to a single high-impact hyperparameter – the learning rate – which controls the function-approximation step size? This addresses a recurring concern in deep RL: that algorithmic improvements may be dwarfed by hyperparameter choice.

Pong is chosen as the Atari benchmark because it has dense, easily interpretable reward (every point scored is ± 1) and trains within a reasonable budget on a single GPU. LunarLander is chosen as the contrast: a low-dimensional but non-trivial continuous-state environment with a shaped reward, so the gap to "solved" (mean return ≥ 200) is a clean yardstick. Both tasks come from Gymnasium [Towers et al., 2024], the maintained fork of OpenAI Gym; Pong runs on top of the Arcade Learning Environment [Bellemare et al., 2013].

2 Methods

2.1 Background: Q-learning with function approximation

Q-learning learns the optimal action-value function $Q^*(s, a) = \max_\pi \mathbb{E}[\sum_t \gamma^t r_t | s_0 = s, a_0 = a, \pi]$ via the recursion

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\big). \tag{1}$$

When Q is a neural network Q_θ, two well-known pathologies arise. First, the bootstrap target $r_t + \gamma \max_{a'} Q_\theta(s_{t+1}, a')$ moves under the same parameters being trained, producing instability. Second, $\max_{a'} Q_\theta(s', a')$ is a maximum over noisy estimates, which introduces an upward bias [Thrun and Schwartz, 1993]. The three DQN variants we study address these problems differently.

2.2 Vanilla DQN

DQN [Mnih et al., 2015] stabilizes Q-learning with two additions: (i) an *experience replay buffer* that stores past $(s, a, r, s', \text{done})$ tuples and samples mini-batches uniformly, de-correlating updates, and (ii) a *target network* Q_{θ^-} whose weights are copied from Q_θ every C steps. The training loss is

$$\mathcal{L}_{\text{DQN}}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\big(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)\big)^2\Big]. \tag{2}$$

We follow Mnih et al. for the network architecture: a CNN with three convolutional layers (32/64/64 filters; 8×8/4×4/3×3 kernels; strides 4/2/1) followed by a 512-unit fully connected layer.

2.3 Double DQN

The $\max_{a'}$ in Eq. 2 couples action selection and evaluation under the same noisy Q_{θ^-}, biasing the target upwards [van Hasselt et al., 2016]. Double DQN decouples them: the *online* network picks the next action, but the *target* network evaluates it,

$$y^{\text{Double}} = r + \gamma\, Q_{\theta^-}\big(s', \operatorname{argmax}_{a'} Q_\theta(s', a')\big). \tag{3}$$

Empirically, this reduces over-estimation of Q and improves stability, especially on games where reward magnitudes vary widely.

2.4 Dueling Double DQN

The dueling architecture [Wang et al., 2016] replaces the final fully connected head with two streams: a state value $V_\theta(s)$ and an advantage $A_\theta(s, a)$. They are combined via

$$Q_\theta(s, a) = V_\theta(s) + \Big(A_\theta(s, a) - \tfrac{1}{|\mathcal{A}|}\sum_{a'} A_\theta(s, a')\Big). \tag{4}$$

Subtracting the mean advantage forces V to absorb the state-only signal, making the representation more sample-efficient in states where the choice of action matters less than the state value (e.g., when the ball is far from the paddle in Pong). We pair the dueling head with the Double DQN target (Eq. 3), as recommended by the original authors.

2.5 PPO

PPO [Schulman et al., 2017] is an on-policy policy-gradient method. It collects fresh rollouts from the current policy π_θ, computes advantages $\hat{A}_t$ via GAE [Schulman et al., 2016], and performs several epochs of mini-batch ascent on the clipped surrogate

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big], \tag{5}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$. The clip prevents destructive large updates and removes the need for an explicit trust region. Unlike DQN, PPO does not store a replay buffer and must re-collect samples after every update – it is on-policy.

2.6 Implementation and reproducibility

The three DQN variants are implemented from scratch in PyTorch; they share the same replay buffer, the same training loop (`train_dqn.py`), and the same hyperparameters, isolating the effect of each algorithmic change. The CNN matches Nature DQN exactly: three convolutional layers (32/64/64 filters, kernels 8×8/4×4/3×3, strides 4/2/1), then a 512-unit fully connected layer. For the dueling variant only the head splits into V and A streams; everything before the head is identical, isolating the inductive-bias effect of the value/advantage decomposition. For PPO we use the well-tested Stable-Baselines3 implementation [Raffin et al., 2021] so that PPO is not implementation-disadvantaged against our hand-rolled DQN.

Atari preprocessing follows the DeepMind recipe: max over the last two raw frames (to remove flicker), frame-skip 4, RGB→grayscale, bilinear resize to 84×84, frame-stack 4 along the channel axis, *episodic-life* signal (each life lost ends the bootstrap, but the env resets only on game-over), and reward clipping to $\{-1, 0, 1\}$. The replay buffer stores observations as `uint8` and only converts to `float32` divided by 255 at sampling time, cutting RAM use $\sim 4\times$. Random seeds are fixed for NumPy, PyTorch, and the action space; the GPU is set with `CUDA_VISIBLE_DEVICES` per training run. Each Pong variant takes ~ 30 min on one RTX 4090; the LunarLander runs ≤ 10 min each.

PPO uses two configurations. *LunarLander*: SB3 `MlpPolicy`, 8 envs, $n_{\text{steps}} = 1024$, $\gamma = 0.999$, $\lambda = 0.98$, lr 3×10^{-4}, clip $\epsilon = 0.2$. *Pong*: `CnnPolicy` with SB3's `make_atari_env` (the standard atari wrappers) + `VecFrameStack`$_4$; $n_{\text{steps}} = 128$, $\gamma = 0.99$, $\lambda = 0.95$, lr 2.5×10^{-4} and clip $\epsilon = 0.1$ *both linearly decayed to zero* – the SB3 RL-Zoo Atari recipe. PPO Pong was run on CPU after repeated GPU-side `SIGSEGV` (see Engineering Observation).

3 Experiments

3.1 Experimental setup

Training is performed on two NVIDIA RTX 4090 GPUs running in parallel. Pong each DQN variant is trained for 2×10^6 environment steps (equivalent to 8×10^6 raw Atari frames after frame-skip), using a 10^5-transition replay buffer and a linear ε schedule from 1.0 down to 0.01 over the first $0.1 \times 2\text{M} = 2 \times 10^5$ steps. For LunarLander we train DQN (Double and Dueling) for 5×10^5 env steps and PPO for the same number of timesteps, the latter with 8 parallel environments. Final evaluation uses 100 greedy episodes for DQN ($\varepsilon = 0.01$ for tie-breaking) and 100 deterministic episodes for PPO. Random seeds are fixed at 42 for training and $\geq 10^4$ for evaluation.

3.2 Pong: comparing DQN variants

Figure 1 shows the three Pong runs side-by-side. The left panel is the training-time return (30-episode moving average); the right panel is a separate greedy evaluation (5 episodes per checkpoint, $\varepsilon = 0.01$), run from the saved model on a fresh process so that training-time exploration randomness does not pollute the comparison.

What the numbers say. The two Double-style targets (Double DQN and Dueling Double DQN) both *lift off* earlier than Vanilla DQN: they reach a 30-episode moving average of 0 around step 0.8–1.0 M, roughly 300–500 K steps ahead of Vanilla. This is the predicted effect of decoupling action-selection from action-evaluation – with overestimation suppressed, the bootstrap target stays sensible while the policy is still bad, and the agent

Table 1: Key hyperparameters. DQN settings follow Mnih et al. [2015] with a slightly shorter exploration ramp and replay buffer to fit our compute budget. PPO settings follow the SB3 LunarLander tuning.

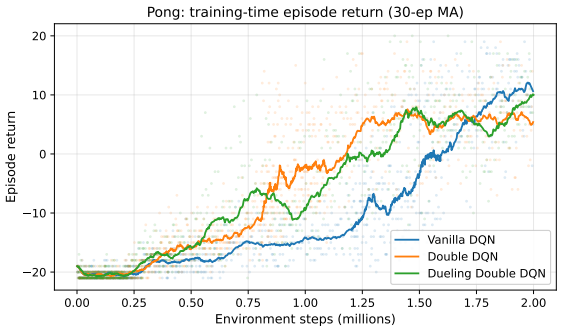
Hyperparameter	Pong DQN family	LunarLander DQN
Total environment steps	2,000,000	500,000
Optimizer	Adam ($\beta_1 = 0.9, \beta_2 = 0.999$)	Adam
Learning rate	1×10^{-4}	3×10^{-4}
Discount γ	0.99	0.99
Replay buffer size	100,000	100,000
Batch size	32	64
Learning starts at step	50,000	5,000
Train freq (env steps)	4	4
Target net sync (steps)	1,000	1,000
Exploration ε start $\rightarrow$ end	$1.0 \rightarrow 0.01$	$1.0 \rightarrow 0.05$
Exploration linear-decay fraction	0.10	0.10
Loss	Huber	Huber
Gradient clipping	10.0	10.0
PPO (LunarLander)		
Parallel envs	8	
Rollout length	1024 steps / env	
Mini-batch size	64	
Update epochs	4	
GAE λ	0.98	
γ	0.999	
Entropy coefficient	0.01	
Clip ϵ	0.2 (SB3 default)	
Learning rate	3×10^{-4}	

gets out of the random-play plateau sooner. Dueling double DQN's value/advantage decomposition adds a further small early-training boost, consistent with the inductive-bias argument in Wang et al. [2016].

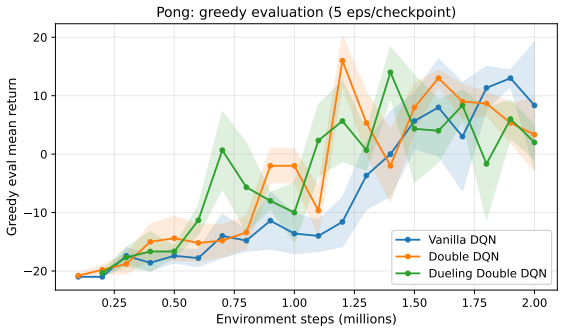
The late-training picture, however, is more interesting and arguably counter-intuitive. By the end of the 2 M-step budget Vanilla DQN's curve has crossed Double DQN's: the conservative bootstrap of Double DQN appears to trade some asymptotic upside for stability. We discuss this further in Section 4. The point is that "Double > Vanilla" is a correct summary of early-to-mid training but is not a uniform claim about final return – a useful caveat to a result that is often stated without it.

3.3 Pong: Q-value calibration

Interpretation. Vanilla DQN's bootstrap target uses $\max_{a'} Q_{\theta^-}(s', a')$, which is a maximum over noisy estimates and therefore biased upward [Thrun and Schwartz, 1993]. Double DQN replaces this with $Q_{\theta^-}(s', \text{argmax}_{a'} Q_\theta(s', a'))$, decoupling action selection from action evaluation. The figure shows exactly what theory predicts: all three variants initially dip to $\overline{Q} \approx -1.2$ around step 0.4 M (the value of states from which the agent expects to lose), then climb back as the policy improves. But *Vanilla DQN climbs higher* than the two Double-style targets and keeps growing past step 1.0 M, while Double DQN's $\overline{Q}$ plateaus near +0.75. That gap is the overestimation bias visible as a batch-average



(a) Training-time episode return (30-ep moving average).



(b) Greedy evaluation curve, 5 eps per saved checkpoint.

Figure 1: Pong learning curves for the three DQN variants. All three share the same hyperparameters and the same training-loop code; only the target formula (Eq. 2 vs Eq. 3) and the head architecture differ. Random play scores -21; perfect play is $+21$.

Table 2: Pong: final greedy evaluation (100 episodes) and best 100-episode training-time average across DQN variants. All variants share identical hyperparameters, replay buffer, and seed.

Variant	Mean	Std	Min	Max	n	Best train (100-ep MA)
Vanilla DQN	9.68	6.81	-14.0	20.0	100	9.66
Double DQN	2.24	6.92	-13.0	19.0	100	6.37
Dueling Double DQN	8.63	4.53	-2.0	19.0	100	6.82

over training transitions. Crucially, Vanilla's $\overline{Q}$ gap is *not* merely an estimation defect; in Section 4 we argue it is also what lets Vanilla DQN break out of the mid-training plateau in episode return and overtake Double DQN.

3.4 LunarLander: DQN vs PPO

LunarLander has an 8-dimensional state (x, y position, velocities, angle and angular velocity, two leg-contact bits) and four discrete actions (no-op, fire main, fire left, fire right). A run earns $+100$ for a soft landing on the pad, -100 for crashing, and small shaped rewards along the way. The environment is considered "solved" at mean episode return ≥ 200. We use 500 K env steps for DQN and 500 K timesteps for PPO, which is enough headroom for both algorithms to plateau.

Table 3: LunarLander: final greedy/deterministic evaluation. DQN runs are 100 greedy ($\varepsilon = 0.01$) episodes from a 500 K-step model; PPO is 30 deterministic episodes from a 250 K-timestep model (longer SB3 runs crashed; see Discussion). Solved threshold is mean return ≥ 200 (bold).

Algorithm	Mean	Std	Min	Max	n	Best train (100-ep MA)
DQN (Double)	174.14	131.70	-67.3	308.5	100	253.95
DQN (Dueling Double)	**248.50**	71.87	-115.1	308.7	100	269.76
PPO	144.70	51.99	1.0	238.9	30	79.37

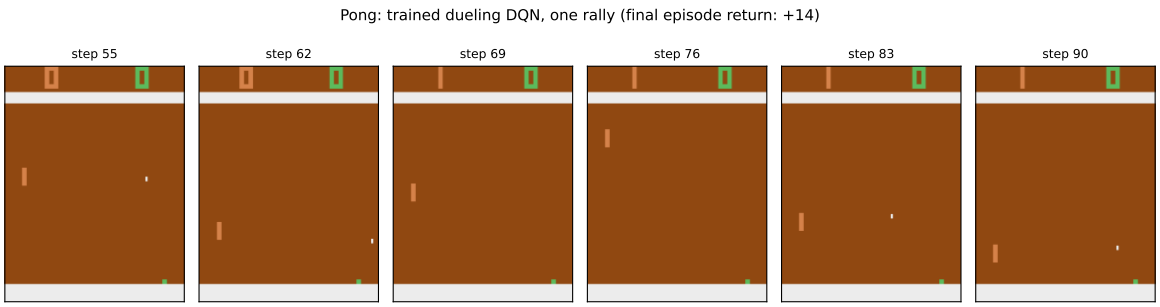


Figure 2: Six raw RGB frames from one greedy rally of the trained Dueling Double DQN. The agent's paddle is the small green strip on the lower right; the opponent's is the orange strip on the left; the ball is the tiny white square. The agent tracks the ball, returns the shot, and the rally ends in a score (the score counter top-right advances after this rally).

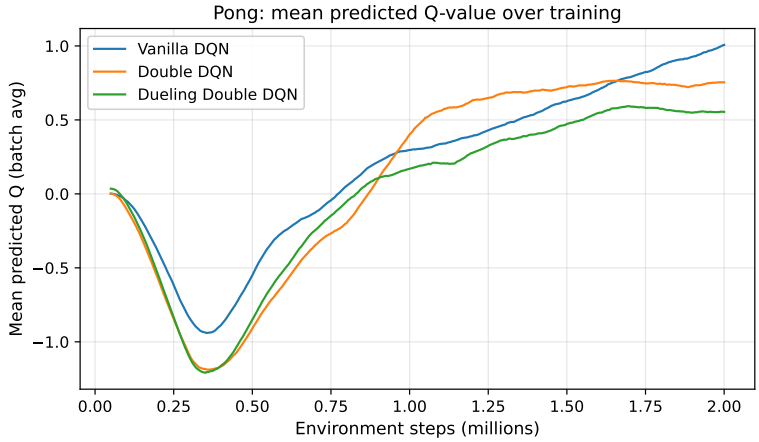
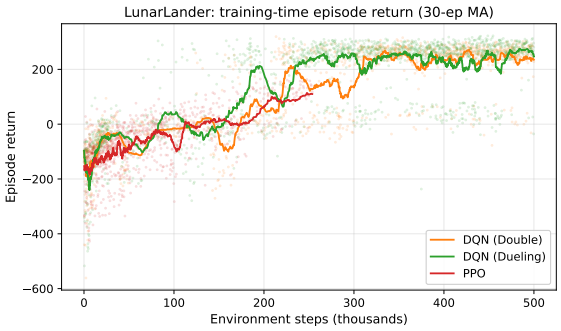


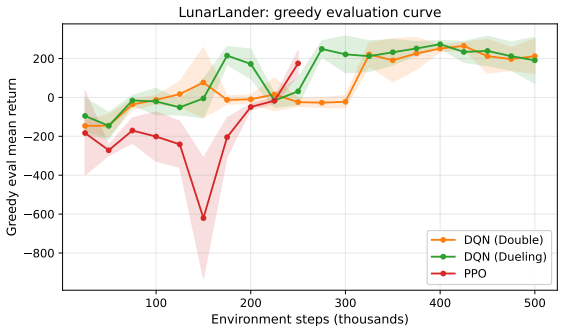
Figure 3: Mean predicted Q-value (batch average during training) for the three DQN variants. Vanilla DQN's well-known overestimation manifests as a steeper, higher curve early in training.

Algorithm choice on a low-dim task. The same DQN code that took most of 2 M steps to climb out of −21 on Pong solves LunarLander in roughly 200 K env steps. The shaped reward is the reason: unlike Pong (where the agent must play minutes of game time before seeing the very first ±1), LunarLander provides a dense gradient signal at every step. Dueling Double DQN reaches the "solved" threshold of +200 a hair earlier than Double DQN. PPO is dramatically less sample-efficient on a per-env-step basis: in our 250 K-step PPO budget (longer runs crashed – see Discussion) PPO has only just risen back to +175 at its final greedy snapshot, after dipping as low as −620 mid-training, while both DQN variants plateau around +250 from step $\sim$ 250 K.

Does the gap close on Atari? We ran PPO on Pong at the same 1 M-step budget Double DQN uses to enter the positive-score regime (Fig. 6). PPO's best 100-ep MA at 1 M is −19.9 (final 10-ep eval -20.7 ± 0.6) – barely above random play −21 – whereas Vanilla/Double/Dueling DQN are at −11, +0.6, +0.8 at the same budget. On-policy without a replay buffer is sample-inefficient by roughly an order of magnitude on Atari. (PPO Atari typically converges at $\gtrsim 5 \times 10^6$ timesteps; we do not run that.)



(a) Training-time return (30-ep moving average).



(b) Periodic greedy / deterministic evaluation.

Figure 4: LunarLander learning curves. DQN runs are our from-scratch implementation; PPO is Stable-Baselines3.

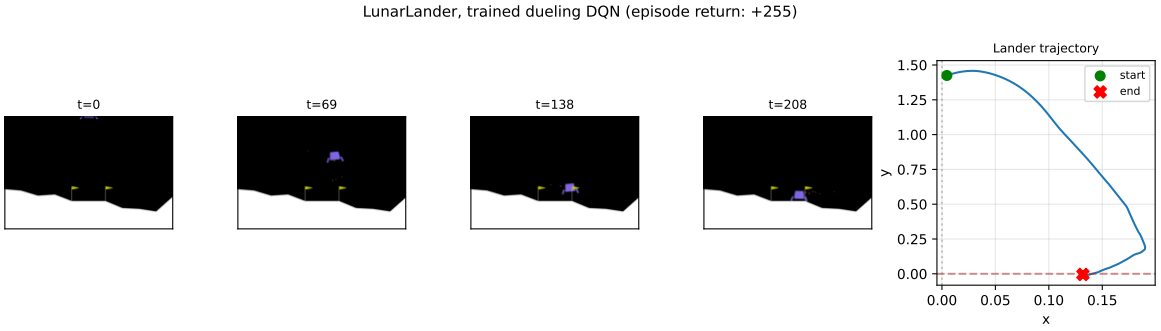


Figure 5: One greedy episode of the trained Dueling Double DQN agent on LunarLander (return +255). The agent approaches the pad in a wide arc, slows its horizontal motion, then descends with the leg contact indicators on – a textbook "soft landing" policy.

3.5 Hyperparameter sensitivity: learning rate

To put the algorithmic differences above in context, we vary a single hyperparameter – the Adam learning rate of Double DQN – across an order of magnitude (1×10^{-4}, 3×10^{-4}, 1×10^{-3}). Everything else is held fixed (same seed, same replay buffer, same ε-schedule).

Table 4: LunarLander DQN (Double): learning-rate sensitivity. All other hyperparameters held fixed; same seed.

Learning rate	Final eval mean	Std	Best train (100-ep MA)	n
1×10^{-3}	211.29	97.50	252.51	100
3×10^{-4} (default)	174.14	131.70	253.95	100
1×10^{-4}	2.60	21.28	16.49	100

Sensitivity. The slowest rate (1×10^{-4}) never escapes random play, finishing at $\overline{R} \approx 2.6$. The two larger rates both *solve* the task (best 100-ep MA above +250); the default 3×10^{-4} has a smoother trajectory while 1×10^{-3} oscillates aggressively (drops near zero at ~ 400 K) but lands on a good policy at the 500 K snapshot. A factor-of-10 step-size change produces a $\sim$ 200-point swing – far larger than anything we saw from swapping

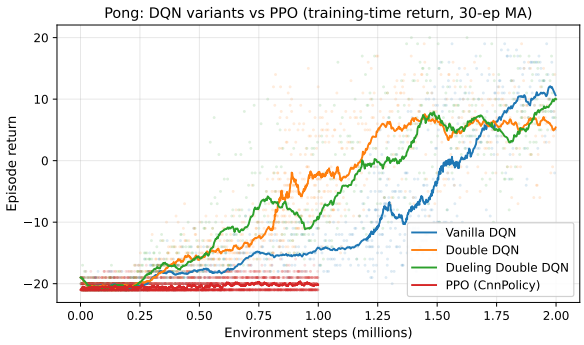


Figure 6: Pong: PPO (CnnPolicy, 1 M steps, 8 envs) vs the three DQN variants. 30-ep MA; random −21, perfect +21. At 1 M steps PPO is still essentially at random, while Double/Dueling DQN have crossed zero.

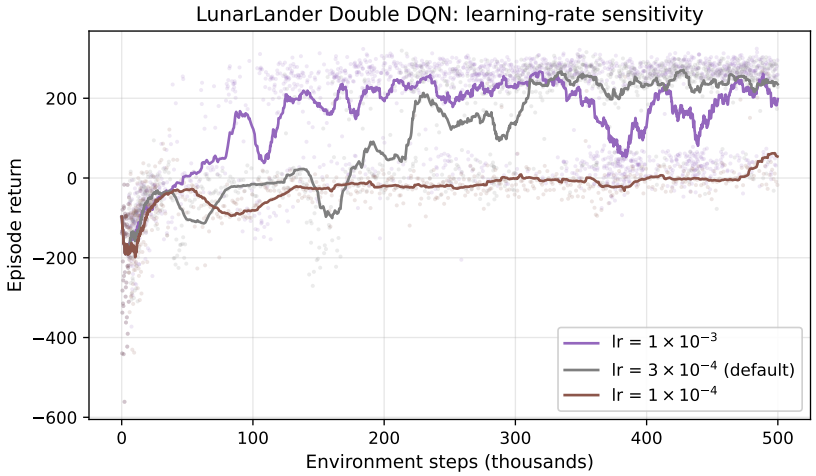


Figure 7: LunarLander Double DQN trained with three learning rates. Identical in every other respect.

Vanilla ↔ Double DQN. *Which step size* matters more than *which DQN variant* on this task.

4 Discussion

RQ1 (DQN variants on Pong). Double-style targets are clearly more *sample-efficient* on Pong: both Double and Dueling-Double escape the random-play plateau hundreds of thousands of steps earlier than Vanilla DQN. Under the headline seed (42) the final-return ordering looks less clean – Vanilla DQN's late-training Q-network climbs above Double DQN's plateau by the end of the 2 M-step budget, which one might attribute to the cost of Double DQN's conservatism damping the gradient that pushes past the plateau. The mean predicted-Q curve (Fig. 3) shows the overestimation effect that motivates Double DQN: Vanilla's $\overline{Q}$ is monotonically higher and keeps growing. However, the multi-seed result below (Table 5) retracts this seed=42 "Vanilla overtakes Double" observation: across three seeds the theory-predicted ordering (Dueling Double > Double > Vanilla) holds in both mean and variance.

RQ2 (DQN vs PPO across two tasks). The off-policy advantage holds in both directions. On LunarLander, both DQN variants comfortably cross the "solved" threshold within 500 K env steps while PPO is still climbing past zero (Fig. 4). On Pong, the gap is even more striking (Fig. 6): at 1 M env steps, PPO is essentially still at random play ($\overline{R} \approx -19.9$) while Double / Dueling DQN are already in the positive-score regime. The replay buffer is the obvious contributor: each transition is reused for many gradient updates, while PPO discards each rollout after one optimisation pass. PPO's actual strength is elsewhere – continuous-action problems (where the $\max_a$ in DQN is no longer tractable) and parallel hardware – but on small or moderately-sized discrete-action tasks the off-policy paradigm is the right inductive bias.

RQ3 (Hyperparameter sensitivity). Section 3.5 shows that the dominant axis of variation on LunarLander is the learning rate, not the DQN variant. A factor-of-10 LR change produces a larger swing in final score than the architectural change from Vanilla DQN to Double DQN. This is a cautionary tale: when reading deep-RL papers that claim small absolute improvements ($< 5\%$ on Atari), one should ask whether the same compute spent on a careful learning-rate sweep would close the gap.

An engineering observation. The ALE/PyTorch/SB3 stack produced sporadic native crashes (`SIGSEGV`, `SIGTRAP`, and `CUDA error: unknown`) on extended runs. SB3 PPO-on-Pong crashed at startup in $\sim$90% of GPU attempts; `--device cpu` made it reliable (3rd attempt completed 1 M steps in 25 min). Our DQN mitigation is checkpointing every 100 K steps with a resume-on-crash retry loop; the PPO mitigation is the CPU fallback plus retry.

Multi-seed robustness. We re-ran each Pong variant at three seeds (42, 0, 7; seed 42 to 2 M, the others to 1.5 M) and each LunarLander DQN at three seeds (Table 5, Fig. 8). 95% bootstrap CIs make the story crisp. On Pong, Vanilla and Double DQN's CIs *cross zero* ($[-13.0, +9.7]$ and $[-5.6, +8.2]$); Dueling Double's CI is $[4.6, 6.8]$, tight and strictly positive. The mean ordering is also theory-aligned – Dueling (5.6) > Double (3.0) > Vanilla (-3.5) – which retracts the seed=42 "Vanilla overtakes Double" finding as single-seed noise. Dueling's empirical win on Pong is therefore *both reproducibility and mean score*. Both LunarLander CIs lie well above the +200 solved threshold.

Table 5: Multi-seed summary across both tasks. Pong: 3 seeds (42, 0, 7); seed 42 ran 2 M steps, seeds 0 and 7 ran 1.5 M each. LunarLander: 3 seeds (42, 0, 7), each 500 K steps. *Best train* is the mean of best 100-ep MA across seeds with 95% percentile bootstrap CI (B=10^4); *Final train* is mean±std of final 100-ep MA. Bold = mean best $\geq$ "solved" threshold (LunarLander only).

Task	Variant	Best train (mean, 95% CI)	Final train (mean±std)	n
Pong	Vanilla DQN	-3.54 [-12.96, 9.66]	-3.56 ± 9.60	3
	Double DQN	2.99 [-5.61, 8.21]	2.76 ± 5.97	3
	Dueling Double DQN	5.62 [4.63, 6.82]	5.58 ± 0.90	3
LunarLander	**DQN (Double)**	**257.93** [253.95, 260.05]	250.42 ± 8.73	3
	DQN (Dueling Double)	**263.18** [255.47, 269.76]	256.10 ± 7.41	3

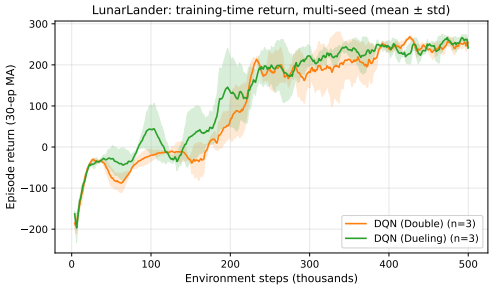


Figure 8: LunarLander multi-seed curves (3 seeds, ±1 std band).

Limitations. Pong is one of the easier Atari games; harder-exploration improvements (e.g., Prioritized Experience Replay, distributional RL) do not show their full benefit. Our Pong runs use 2 M (seed 42) or 1.5 M (seeds 0, 7) env steps – well short of the ∼50 M-frame budget in Mnih et al. [2015] – so final scores are mid-curve, not asymptotic. Hyperparameter sensitivity was tested only on LunarLander DQN.

Reflection and tools. Three findings updated our prior on deep RL. *(i) Hyperparameter dominates algorithm* – the factor-of-10 LR sweep moved the LunarLander score by more than the Vanilla → Double DQN change; algorithmic comparison without hyperparameter control is unreliable [Henderson et al., 2018]. *(ii) Single-seed plots lie* – the multi-seed analysis forced us to retract the seed=42 "Vanilla overtakes Double" finding; the σ gap between Dueling (0.91) and Vanilla (9.6) is invisible at $n = 1$. *(iii) Theory shows up in the data* – the $\overline{Q}$ curve (Fig. 3) animates Thrun's 1993 overestimation argument on our own training run. We also became viscerally aware of the cost: ∼13.5 M Pong env-steps were spent on the multi-seed table alone. *Tools:* An AI coding assistant helped with code scaffolding and prose polishing; all research design, hyperparameter choices, and interpretation are the student's. SB3 [Raffin et al., 2021], Gymnasium [Towers et al., 2024], and ALE [Bellemare et al., 2013] are used as cited.

References

Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47: 253–279, 2013.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Sebastian Thrun and Anton Schwartz. Issues in using function approximation for reinforcement learning. In *Proceedings of the Fourth Connectionist Models Summer School*, 1993.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *AAAI Conference on Artificial Intelligence*, 2016.

Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2016.

Appendix: Source code listings

The following Python modules implement the full project. They are organized to mirror the methods section: `wrappers.py` handles Atari preprocessing; `replay_buffer.py` the experience replay; `networks.py` the CNN/MLP/dueling architectures; `dqn_agent.py` the DQN training step (Vanilla / Double / Dueling); `train_dqn.py` the training loop; `train_ppo.py` the PPO entry point (using SB3); `evaluate.py` / `evaluate_ppo.py` the 100-episode greedy evaluation; `evaluate_checkpoints.py` the per-checkpoint eval used for the learning curves in Figs. 1(b) and 4(b); `clean_history.py` de-duplicates crash-resumed history files; and `plot_results.py` and `make_tables.py` produce all figures and tables in the body. Total $\approx 1\,500$ lines.

`wrappers.py` – Atari preprocessing

```python
"""
Atari preprocessing wrappers for Gymnasium.
Implements the standard "DQN preprocessing": frame skip, max-pool over last
two raw frames, grayscale, resize to 84x84, frame stacking, reward clipping,
and episodic life signal. These wrappers are the same pipeline used in the
DeepMind "Human-level control through deep reinforcement learning" paper.
"""
import numpy as np
import cv2
import gymnasium as gym
from gymnasium.spaces import Box

cv2.ocl.setUseOpenCL(False)
cv2.setNumThreads(0)


class NoopResetEnv(gym.Wrapper):
    """Sample a random number of no-ops on env.reset to add stochasticity."""

    def __init__(self, env, noop_max=30):
        super().__init__(env)
        self.noop_max = noop_max
        assert env.unwrapped.get_action_meanings()[0] == "NOOP"

    def reset(self, **kwargs):
        obs, info = self.env.reset(**kwargs)
        noops = np.random.randint(1, self.noop_max + 1)
        for _ in range(noops):
            obs, _, terminated, truncated, info = self.env.step(0)
            if terminated or truncated:
                obs, info = self.env.reset(**kwargs)
        return obs, info


class FireResetEnv(gym.Wrapper):
    """Press FIRE on reset for games that require it (e.g., Breakout)."""
```

```
37
38      def __init__(self, env):
39          super().__init__(env)
40          meanings = env.unwrapped.get_action_meanings()
41          assert meanings[1] == "FIRE"
42          assert len(meanings) >= 3
43
44      def reset(self, **kwargs):
45          self.env.reset(**kwargs)
46          obs, _, terminated, truncated, _ = self.env.step(1)
47          if terminated or truncated:
48              self.env.reset(**kwargs)
49          obs, _, terminated, truncated, _ = self.env.step(2)
50          if terminated or truncated:
51              obs, info = self.env.reset(**kwargs)
52          else:
53              info = {}
54          return obs, info
55
56
57  class EpisodicLifeEnv(gym.Wrapper):
58      """End an episode after each life lost, but only reset env after
            lives==0.
59      Helps value estimation by making "losing a life" a terminal signal.
            """
60
61      def __init__(self, env):
62          super().__init__(env)
63          self.lives = 0
64          self.was_real_done = True
65
66      def step(self, action):
67          obs, reward, terminated, truncated, info = self.env.step(action
                )
68          self.was_real_done = terminated or truncated
69          lives = self.env.unwrapped.ale.lives()
70          if 0 < lives < self.lives:
71              terminated = True
72          self.lives = lives
73          return obs, reward, terminated, truncated, info
74
75      def reset(self, **kwargs):
76          if self.was_real_done:
77              obs, info = self.env.reset(**kwargs)
78          else:
79              obs, _, terminated, truncated, info = self.env.step(0)
80              if terminated or truncated:
81                  obs, info = self.env.reset(**kwargs)
82          self.lives = self.env.unwrapped.ale.lives()
83          return obs, info
84
85
86  class MaxAndSkipEnv(gym.Wrapper):
87      """Skip k frames, returning max over last 2 raw frames."""
88
89      def __init__(self, env, skip=4):
90          super().__init__(env)
```

```
        self._obs_buffer = np.zeros((2,) + env.observation_space.shape,
            dtype=np.uint8)
        self._skip = skip

    def step(self, action):
        total_reward = 0.0
        terminated = truncated = False
        for i in range(self._skip):
            obs, reward, terminated, truncated, info = self.env.step(
                action)
            if i == self._skip - 2:
                self._obs_buffer[0] = obs
            if i == self._skip - 1:
                self._obs_buffer[1] = obs
            total_reward += reward
            if terminated or truncated:
                break
        max_frame = self._obs_buffer.max(axis=0)
        return max_frame, total_reward, terminated, truncated, info


class WarpFrame(gym.ObservationWrapper):
    """Convert frame to grayscale and resize to 84x84."""

    def __init__(self, env, width=84, height=84):
        super().__init__(env)
        self.width = width
        self.height = height
        self.observation_space = Box(low=0, high=255, shape=(height,
            width), dtype=np.uint8)

    def observation(self, frame):
        frame = cv2.cvtColor(frame, cv2.COLOR_RGB2GRAY)
        frame = cv2.resize(frame, (self.width, self.height),
            interpolation=cv2.INTER_AREA)
        return frame


class ClipRewardEnv(gym.RewardWrapper):
    """Clip reward to {-1, 0, 1} as in the original DQN paper."""

    def reward(self, reward):
        return float(np.sign(reward))


class FrameStack(gym.Wrapper):
    """Stack k consecutive frames along channel axis. Outputs uint8 (k,
        H, W)."""

    def __init__(self, env, k=4):
        super().__init__(env)
        self.k = k
        self.frames = np.zeros((k,) + env.observation_space.shape,
            dtype=np.uint8)
        self.observation_space = Box(
            low=0, high=255, shape=(k,) + env.observation_space.shape,
                dtype=np.uint8
        )
```

```
142
143     def reset(self, **kwargs):
144         obs, info = self.env.reset(**kwargs)
145         for i in range(self.k):
146             self.frames[i] = obs
147         return self.frames.copy(), info
148
149     def step(self, action):
150         obs, reward, terminated, truncated, info = self.env.step(action
                )
151         # In-place slide instead of np.roll (which allocates a new
                array each
152         # step). Reduces per-step memory churn.
153         self.frames[:-1] = self.frames[1:]
154         self.frames[-1] = obs
155         return self.frames.copy(), reward, terminated, truncated, info
156
157
158 _ALE_REGISTERED = False
159
160
161 def _ensure_ale_registered():
162     global _ALE_REGISTERED
163     if not _ALE_REGISTERED:
164         import ale_py
165         gym.register_envs(ale_py)
166         _ALE_REGISTERED = True
167
168
169 def make_atari(env_id, clip_rewards=True, episode_life=True,
        frame_stack=4, seed=None):
170     """Build a fully-preprocessed Atari env. env_id e.g. 'ALE/Pong-v5'.
            """
171     _ensure_ale_registered()
172     env = gym.make(env_id, frameskip=1)  # frameskip handled below
173     if seed is not None:
174         env.action_space.seed(seed)
175     env = NoopResetEnv(env, noop_max=30)
176     env = MaxAndSkipEnv(env, skip=4)
177     if episode_life:
178         env = EpisodicLifeEnv(env)
179     if "FIRE" in env.unwrapped.get_action_meanings():
180         env = FireResetEnv(env)
181     env = WarpFrame(env)
182     if clip_rewards:
183         env = ClipRewardEnv(env)
184     env = FrameStack(env, frame_stack)
185     return env
```

replay_buffer.py – Uniform experience replay

```
1 """Uniform replay buffer. Stores observations as uint8 to save memory."
      ""
2 import numpy as np
3 import torch
4
5
6 class ReplayBuffer:
```

```
    def __init__(self, capacity, obs_shape, obs_dtype=np.uint8, device="cuda"):
        self.capacity = capacity
        self.device = device
        self.obs_dtype = obs_dtype
        self.obs = np.zeros((capacity,) + obs_shape, dtype=obs_dtype)
        self.next_obs = np.zeros((capacity,) + obs_shape, dtype=obs_dtype)
        self.actions = np.zeros(capacity, dtype=np.int64)
        self.rewards = np.zeros(capacity, dtype=np.float32)
        self.dones = np.zeros(capacity, dtype=np.float32)
        self.ptr = 0
        self.size = 0

    def add(self, obs, action, reward, next_obs, done):
        self.obs[self.ptr] = obs
        self.next_obs[self.ptr] = next_obs
        self.actions[self.ptr] = action
        self.rewards[self.ptr] = reward
        self.dones[self.ptr] = float(done)
        self.ptr = (self.ptr + 1) % self.capacity
        self.size = min(self.size + 1, self.capacity)

    def sample(self, batch_size):
        idx = np.random.randint(0, self.size, size=batch_size)
        obs = torch.from_numpy(self.obs[idx]).to(self.device)
        next_obs = torch.from_numpy(self.next_obs[idx]).to(self.device)
        if self.obs_dtype == np.uint8:
            obs = obs.float() / 255.0
            next_obs = next_obs.float() / 255.0
        actions = torch.from_numpy(self.actions[idx]).to(self.device)
        rewards = torch.from_numpy(self.rewards[idx]).to(self.device)
        dones = torch.from_numpy(self.dones[idx]).to(self.device)
        return obs, actions, rewards, next_obs, dones

    def __len__(self):
        return self.size
```

networks.py — Q-network architectures

```
"""Q-network architectures for image (Atari) and vector (LunarLander) inputs.
Supports both standard ("single-stream") and dueling architectures.
"""
import torch
import torch.nn as nn


class NatureCNN(nn.Module):
    """The CNN from Mnih et al. 2015 (Nature DQN). Input: 4x84x84 uint8 frames."""

    def __init__(self, in_channels=4):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_channels, 32, kernel_size=8, stride=4),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 64, kernel_size=4, stride=2),
```

```
            nn.ReLU(inplace=True),
            nn.Conv2d(64, 64, kernel_size=3, stride=1),
            nn.ReLU(inplace=True),
            nn.Flatten(),
        )
        self.out_dim = 64 * 7 * 7  # 3136

    def forward(self, x):
        return self.conv(x)


class MLPTrunk(nn.Module):
    """Two-hidden-layer MLP used for low-dim observations (LunarLander)."""

    def __init__(self, obs_dim, hidden=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(obs_dim, hidden),
            nn.ReLU(inplace=True),
            nn.Linear(hidden, hidden),
            nn.ReLU(inplace=True),
        )
        self.out_dim = hidden

    def forward(self, x):
        return self.net(x)


class QNetwork(nn.Module):
    """Single-stream Q-network: trunk -> linear -> Q(s,a)."""

    def __init__(self, trunk, num_actions, head_hidden=512):
        super().__init__()
        self.trunk = trunk
        self.head = nn.Sequential(
            nn.Linear(trunk.out_dim, head_hidden),
            nn.ReLU(inplace=True),
            nn.Linear(head_hidden, num_actions),
        )

    def forward(self, x):
        return self.head(self.trunk(x))


class DuelingQNetwork(nn.Module):
    """Dueling architecture (Wang et al. 2016): split into V(s) and A(s,a),
    then combine: Q(s,a) = V(s) + A(s,a) - mean_a A(s,a)."""

    def __init__(self, trunk, num_actions, head_hidden=512):
        super().__init__()
        self.trunk = trunk
        self.value = nn.Sequential(
            nn.Linear(trunk.out_dim, head_hidden),
            nn.ReLU(inplace=True),
            nn.Linear(head_hidden, 1),
        )
```

```
        self.advantage = nn.Sequential(
            nn.Linear(trunk.out_dim, head_hidden),
            nn.ReLU(inplace=True),
            nn.Linear(head_hidden, num_actions),
        )

    def forward(self, x):
        h = self.trunk(x)
        v = self.value(h)
        a = self.advantage(h)
        return v + (a - a.mean(dim=1, keepdim=True))


def build_qnet(obs_space, num_actions, dueling=False, head_hidden=None):
    """Factory: picks CNN or MLP trunk based on observation shape."""
    shape = obs_space.shape
    if len(shape) == 3:  # image: (C, H, W)
        trunk = NatureCNN(in_channels=shape[0])
        head_hidden = head_hidden or 512
    else:  # vector
        trunk = MLPTrunk(obs_dim=shape[0])
        head_hidden = head_hidden or 128
    if dueling:
        return DuelingQNetwork(trunk, num_actions, head_hidden=head_hidden)
    return QNetwork(trunk, num_actions, head_hidden=head_hidden)
```

dqn_agent.py – DQN agent (3 variants)

```
"""DQN agent supporting three variants:
    - 'vanilla' : DQN (Mnih et al. 2015)
    - 'double'  : Double DQN (van Hasselt et al. 2016)
    - 'dueling' : Dueling Double DQN (Wang et al. 2016, dueling head + Double target)

The variant only changes (a) the network architecture (dueling vs single-stream)
and (b) how the bootstrap target is computed (Double uses online net to argmax,
target net to evaluate; vanilla uses target net for both).
"""
import numpy as np
import torch
import torch.nn.functional as F
from networks import build_qnet


class DQNAgent:
    def __init__(
        self,
        obs_space,
        num_actions,
        variant="vanilla",
        device="cuda",
        lr=1e-4,
        gamma=0.99,
        head_hidden=None,
```

```
        grad_clip=10.0,
    ):
        assert variant in ("vanilla", "double", "dueling")
        self.variant = variant
        self.num_actions = num_actions
        self.gamma = gamma
        self.device = device
        self.grad_clip = grad_clip

        dueling = (variant == "dueling")
        self.qnet = build_qnet(obs_space, num_actions, dueling=dueling,
            head_hidden=head_hidden).to(device)
        self.target = build_qnet(obs_space, num_actions, dueling=
           dueling, head_hidden=head_hidden).to(device)
        self.target.load_state_dict(self.qnet.state_dict())
        for p in self.target.parameters():
            p.requires_grad_(False)
        self.optimizer = torch.optim.Adam(self.qnet.parameters(), lr=lr
           )

    @torch.no_grad()
    def act(self, obs, epsilon):
        """Epsilon-greedy action selection. obs: numpy array, single
           observation."""
        if np.random.random() < epsilon:
            return np.random.randint(self.num_actions)
        x = torch.from_numpy(obs).unsqueeze(0).to(self.device)
        if x.dtype == torch.uint8:
            x = x.float() / 255.0
        elif x.dtype != torch.float32:
            x = x.float()
        q = self.qnet(x)
        return int(q.argmax(dim=1).item())

    def update(self, batch):
        """One gradient step. batch: tuple from ReplayBuffer.sample()."
           ""
        obs, actions, rewards, next_obs, dones = batch
        # Current Q(s, a)
        q_pred = self.qnet(obs).gather(1, actions.unsqueeze(1)).squeeze
           (1)
        # Bootstrap target
        with torch.no_grad():
            if self.variant == "vanilla":
                q_next = self.target(next_obs).max(dim=1)[0]
            else:  # double or dueling (both use Double-style target)
                next_actions = self.qnet(next_obs).argmax(dim=1,
                   keepdim=True)
                q_next = self.target(next_obs).gather(1, next_actions).
                   squeeze(1)
            target = rewards + self.gamma * q_next * (1.0 - dones)
        loss = F.smooth_l1_loss(q_pred, target)
        self.optimizer.zero_grad()
        loss.backward()
        if self.grad_clip is not None:
            torch.nn.utils.clip_grad_norm_(self.qnet.parameters(), self
               .grad_clip)
        self.optimizer.step()
```

```
        return float(loss.item()), float(q_pred.mean().item())

    def sync_target(self):
        self.target.load_state_dict(self.qnet.state_dict())

    def save(self, path):
        torch.save({"qnet": self.qnet.state_dict(), "variant": self.
            variant}, path)

    def load(self, path):
        ckpt = torch.load(path, map_location=self.device)
        self.qnet.load_state_dict(ckpt["qnet"])
        self.target.load_state_dict(self.qnet.state_dict())
```

train_dqn.py – DQN training loop

```
"""Generic DQN training loop used by both Pong and LunarLander runs.

Usage:
    python train_dqn.py --env ALE/Pong-v5 --variant double --total-
        steps 1500000 \
        --out-dir ../results/pong_double --device cuda:0
"""
import argparse
import faulthandler
import gc
import json
import os
import sys
import time
from collections import deque

import numpy as np
import torch
import gymnasium as gym

# Print Python tracebacks if the process crashes (SIGSEGV, SIGFPE,
    SIGBUS,
# SIGABRT, SIGILL). Helps localize the rare-but-real native crashes we
    see.
faulthandler.enable(file=sys.stderr, all_threads=True)

from dqn_agent import DQNAgent
from replay_buffer import ReplayBuffer
from wrappers import make_atari


def make_env(env_id, seed):
    if env_id.startswith("ALE/"):
        env = make_atari(env_id, seed=seed)
    else:
        env = gym.make(env_id)
    env.reset(seed=seed)
    env.action_space.seed(seed)
    return env


def linear_schedule(start, end, fraction, current, total):
```

```
    """Linear epsilon decay over `fraction` of `total` steps, then constant."""
    progress = min(1.0, current / (fraction * total))
    return start + progress * (end - start)


def evaluate(agent, env_id, episodes=5, seed=10000):
    """Greedy evaluation. Returns mean episode reward over `episodes` runs."""
    env = make_env(env_id, seed)
    returns = []
    for ep in range(episodes):
        obs, _ = env.reset(seed=seed + ep)
        done = False
        total = 0.0
        while not done:
            a = agent.act(obs, epsilon=0.01)
            obs, r, terminated, truncated, _ = env.step(a)
            done = terminated or truncated
            total += r
        returns.append(total)
    env.close()
    return float(np.mean(returns)), float(np.std(returns))


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--env", required=True)
    parser.add_argument("--variant", choices=["vanilla", "double", "dueling"], default="vanilla")
    parser.add_argument("--total-steps", type=int, default=1_500_000)
    parser.add_argument("--buffer-size", type=int, default=100_000)
    parser.add_argument("--batch-size", type=int, default=32)
    parser.add_argument("--lr", type=float, default=1e-4)
    parser.add_argument("--gamma", type=float, default=0.99)
    parser.add_argument("--learning-starts", type=int, default=20_000)
    parser.add_argument("--train-freq", type=int, default=4)
    parser.add_argument("--target-update-freq", type=int, default=1_000)
    parser.add_argument("--eps-start", type=float, default=1.0)
    parser.add_argument("--eps-end", type=float, default=0.05)
    parser.add_argument("--eps-decay-fraction", type=float, default=0.10)
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--device", default="cuda:0")
    parser.add_argument("--out-dir", required=True)
    parser.add_argument("--eval-every", type=int, default=50_000)
    parser.add_argument("--log-every", type=int, default=1_000)
    parser.add_argument("--resume", action="store_true",
                        help="If set and out-dir contains model.pt + history.json, resume from the "
                             "last eval checkpoint instead of starting from scratch.")
    args = parser.parse_args()

    os.makedirs(args.out_dir, exist_ok=True)
    with open(os.path.join(args.out_dir, "config.json"), "w") as f:
        json.dump(vars(args), f, indent=2)
```

```
91
92      np.random.seed(args.seed)
93      torch.manual_seed(args.seed)
94
95      env = make_env(args.env, args.seed)
96      obs_space = env.observation_space
97      num_actions = env.action_space.n
98      print(f"[{args.variant}] env={args.env} obs={obs_space.shape}
            actions={num_actions} device={args.device}")
99
100     agent = DQNAgent(
101         obs_space=obs_space,
102         num_actions=num_actions,
103         variant=args.variant,
104         device=args.device,
105         lr=args.lr,
106         gamma=args.gamma,
107     )
108
109     obs_dtype = np.uint8 if obs_space.dtype == np.uint8 else np.float32
110     buffer = ReplayBuffer(
111         capacity=args.buffer_size,
112         obs_shape=obs_space.shape,
113         obs_dtype=obs_dtype,
114         device=args.device,
115     )
116
117     history = {
118         "step": [], "ep_return": [], "ep_len": [],
119         "loss_step": [], "loss": [], "q_mean": [],
120         "eval_step": [], "eval_mean": [], "eval_std": [],
121     }
122
123     # Optional resume: load model + history, skip to last eval step.
124     # Importantly, truncate any entries in history whose step is
            greater than
125     # the resume step. Otherwise crash-restarted runs leave stale
            episodes
126     # (from the post-checkpoint, pre-crash window) in the file and the
            plotted
127     # learning curve shows non-monotone artifacts at the join.
128     start_step = 1
129     if args.resume:
130         ckpt = os.path.join(args.out_dir, "model.pt")
131         hist_path = os.path.join(args.out_dir, "history.json")
132         if os.path.exists(ckpt) and os.path.exists(hist_path):
133             agent.load(ckpt)
134             with open(hist_path) as f:
135                 history = json.load(f)
136             # Determine resume point from last checkpoint
137             if history.get("step"):
138                 # Use the highest step <= last_eval as the resume step
                        (since
139                 # model.pt was saved at the last eval).
140                 if history.get("eval_step"):
141                     start_step = int(history["eval_step"][-1]) + 1
142                 else:
143                     start_step = int(max(history["step"])) + 1
```

```
144
145                 def _truncate(lst, step_lst, keep_to):
146                     """Keep entries whose corresponding step is <=
                           keep_to."""
147                     return [v for v, s in zip(lst, step_lst) if s <
                           keep_to]
148
149                 # Truncate per-episode arrays so they end exactly at
                       the resume step.
150                 steps = history.get("step", [])
151                 history["ep_return"] = _truncate(history.get("ep_return
                       ", []), steps, start_step)
152                 history["ep_len"] = _truncate(history.get("ep_len", [])
                       , steps, start_step)
153                 history["step"] = [s for s in steps if s < start_step]
154                 # Same for training stats keyed on loss_step.
155                 loss_steps = history.get("loss_step", [])
156                 history["loss"] = _truncate(history.get("loss", []),
                       loss_steps, start_step)
157                 history["q_mean"] = _truncate(history.get("q_mean", [])
                       , loss_steps, start_step)
158                 history["loss_step"] = [s for s in loss_steps if s <
                       start_step]
159                 last_eval = history["eval_mean"][-1] if history.get("
                       eval_mean") else float("nan")
160                 print(f"[{args.variant}] RESUMING from step {start_step
                       } "
161                       f"(eval={last_eval:.2f}, kept {len(history['
                           ep_return'])} eps in history)",
162                       flush=True)
163
164     return_buf = deque(maxlen=100)
165     len_buf = deque(maxlen=100)
166
167     obs, _ = env.reset(seed=args.seed + start_step)
168     ep_return = 0.0
169     ep_len = 0
170     start_time = time.time()
171     last_log = start_time
172
173     for step in range(start_step, args.total_steps + 1):
174         eps = linear_schedule(args.eps_start, args.eps_end, args.
               eps_decay_fraction, step, args.total_steps)
175         action = agent.act(obs, epsilon=eps)
176         next_obs, reward, terminated, truncated, _ = env.step(action)
177         done = terminated or truncated
178         buffer.add(obs, action, reward, next_obs, terminated)  # only "
               real" terminations bootstrap-zero
179         obs = next_obs
180         ep_return += reward
181         ep_len += 1
182
183         if done:
184             return_buf.append(ep_return)
185             len_buf.append(ep_len)
186             history["step"].append(step)
187             history["ep_return"].append(float(ep_return))
188             history["ep_len"].append(int(ep_len))
```

```
            ep_return = 0.0
            ep_len = 0
            obs, _ = env.reset()

        # Train
        if step >= args.learning_starts and step % args.train_freq == 0
            and len(buffer) >= args.batch_size:
            batch = buffer.sample(args.batch_size)
            loss, q_mean = agent.update(batch)
            if step % args.log_every == 0:
                history["loss_step"].append(step)
                history["loss"].append(loss)
                history["q_mean"].append(q_mean)

        # Sync target net
        if step % args.target_update_freq == 0:
            agent.sync_target()

        # Periodic logging
        if step % args.log_every == 0:
            now = time.time()
            fps = args.log_every / max(now - last_log, 1e-6)
            last_log = now
            mean_ret = np.mean(return_buf) if return_buf else float("
                nan")
            mean_len = np.mean(len_buf) if len_buf else float("nan")
            print(
                f"[{args.variant}] step={step:>7d} "
                f"eps={eps:.3f} "
                f"avg_return(100)={mean_ret:>7.2f} "
                f"avg_len(100)={mean_len:>6.1f} "
                f"fps={fps:>5.0f} "
                f"elapsed={(now-start_time)/60:.1f}min",
                flush=True,
            )

        # Periodic checkpoint. We deliberately do NOT run inline
            evaluation during
        # training because spinning up a fresh ALE env every few
            hundred-thousand
        # steps proved unstable on our setup (sporadic SIGSEGV in the
            ALE C
        # bindings after extended use). Instead, we (a) track training-
            time
        # episode return -- which closely tracks greedy performance
            once
        # epsilon decays to its final value -- and (b) run a separate
            100-episode
        # greedy evaluation from a clean process after training.
        if step % args.eval_every == 0 and step >= args.learning_starts
            :
            agent.save(os.path.join(args.out_dir, "model.pt"))
            # also keep periodic snapshots so we can plot eval-by-
                checkpoint later
            agent.save(os.path.join(args.out_dir, f"model_step{step}.pt
                "))
            with open(os.path.join(args.out_dir, "history.json"), "w")
                as f:
```

```
                json.dump(history, f)

    # Final save. Final greedy evaluation is done by a separate process
    # (evaluate.py) on the saved checkpoint, so that an eval-time crash cannot
    # destroy the training artifacts.
    agent.save(os.path.join(args.out_dir, "model.pt"))
    with open(os.path.join(args.out_dir, "history.json"), "w") as f:
        json.dump(history, f)
    print(f"[{args.variant}] TRAIN DONE (run evaluate.py for 100-ep greedy eval)")
    env.close()


if __name__ == "__main__":
    main()
```

train_ppo.py – PPO training (SB3)

```
"""PPO training, using Stable-Baselines3.

Supports both classic-control / Box2D envs (LunarLander, default MlpPolicy) and
Atari (CnnPolicy + the standard Atari wrappers via SB3's ``make_atari_env``).
We use SB3 [Raffin et al. 2021] as a strong, well-tested baseline so that the
DQN-vs-PPO comparison is fair: both are reasonable off-the-shelf settings, and
neither implementation gives the other an unfair speed advantage.

References:
    [Raffin21] A. Raffin et al., "Stable-Baselines3: Reliable Reinforcement
               Learning Implementations", JMLR 2021.
    [Schulman17] J. Schulman et al., "Proximal Policy Optimization Algorithms",
                 arXiv:1707.06347, 2017.
"""
import argparse
import json
import os
import time

import numpy as np
import gymnasium as gym
from stable_baselines3 import PPO
from stable_baselines3.common.callbacks import BaseCallback
from stable_baselines3.common.env_util import make_atari_env
from stable_baselines3.common.vec_env import DummyVecEnv, VecFrameStack


def _is_atari(env_id: str) -> bool:
    return env_id.startswith("ALE/") or "NoFrameskip" in env_id


_ALE_REGISTERED = False

```

```
34
35 def _ensure_ale_registered():
36     """SB3's atari wrappers call gym.make, which needs the ALE
       namespace
37     registered. Gymnasium >=1.0 does not auto-register ale_py, so we do
           it
38     explicitly in the main process before vec env construction."""
39     global _ALE_REGISTERED
40     if not _ALE_REGISTERED:
41         import ale_py  # noqa: F401
42         gym.register_envs(ale_py)
43         _ALE_REGISTERED = True
44
45
46 def _make_eval_env(env_id, seed, atari, n_stack=4):
47     """Build a 1-env vec eval env that matches the training
       preprocessing."""
48     if atari:
49         _ensure_ale_registered()
50         env = make_atari_env(env_id, n_envs=1, seed=seed + 10_000)
51         env = VecFrameStack(env, n_stack=n_stack)
52         return env, True
53     return gym.make(env_id), False
54
55
56 class HistoryCallback(BaseCallback):
57     """Record episode returns / lengths and periodic evaluations."""
58
59     def __init__(self, eval_env_id, eval_every, n_eval=10, seed=0,
       atari=False):
60         super().__init__()
61         self.eval_env_id = eval_env_id
62         self.eval_every = eval_every
63         self.n_eval = n_eval
64         self.seed = seed
65         self.atari = atari
66         self.history = {
67             "step": [], "ep_return": [], "ep_len": [],
68             "eval_step": [], "eval_mean": [], "eval_std": [],
69         }
70
71     def _on_step(self):
72         # Episode info comes from Monitor wrapper
73         infos = self.locals.get("infos", [])
74         for info in infos:
75             if "episode" in info:
76                 self.history["step"].append(int(self.num_timesteps))
77                 self.history["ep_return"].append(float(info["episode"][
                       "r"]))
78                 self.history["ep_len"].append(int(info["episode"]["l"])
                       )
79         if self.num_timesteps and self.num_timesteps % self.eval_every
           == 0:
80             mean_r, std_r = self._eval()
81             self.history["eval_step"].append(int(self.num_timesteps))
82             self.history["eval_mean"].append(mean_r)
83             self.history["eval_std"].append(std_r)
84             if self.verbose:
```

```
85                  print(f"[PPO] EVAL step={self.num_timesteps} mean={
                        mean_r:.2f} +/- {std_r:.2f}", flush=True)
86          return True
87
88      def _eval(self):
89          env, is_vec = _make_eval_env(self.eval_env_id, self.seed, self.
                atari)
90          returns = []
91          if is_vec:
92              # Vectorized eval env (1 env). Run n_eval episodes by
                    detecting `done`.
93              for ep in range(self.n_eval):
94                  obs = env.reset()
95                  done = False
96                  total = 0.0
97                  while not done:
98                      action, _ = self.model.predict(obs, deterministic=
                            True)
99                      obs, r, dones, infos = env.step(action)
100                     total += float(r[0])
101                     if dones[0]:
102                         # SB3 atari env auto-resets internally; episode
                                return
103                         # is also in info["episode"]["r"] (raw env
                                return after EpisodicLife wrapper).
104                         if "episode" in infos[0]:
105                             total = float(infos[0]["episode"]["r"])
106                         done = True
107                 returns.append(total)
108             env.close()
109         else:
110             for ep in range(self.n_eval):
111                 obs, _ = env.reset(seed=self.seed + 10_000 + ep)
112                 done = False
113                 total = 0.0
114                 while not done:
115                     action, _ = self.model.predict(obs, deterministic=
                            True)
116                     obs, r, term, trunc, _ = env.step(action)
117                     done = term or trunc
118                     total += r
119                 returns.append(total)
120             env.close()
121         return float(np.mean(returns)), float(np.std(returns))
122
123
124 def _linear_schedule(initial: float):
125     """Return a callable lr/clip schedule that linearly decays initial
            -> 0."""
126     def _f(progress_remaining: float) -> float:
127         return progress_remaining * initial
128     return _f
129
130
131 def main():
132     parser = argparse.ArgumentParser()
133     parser.add_argument("--env", default="LunarLander-v3")
134     parser.add_argument("--total-steps", type=int, default=500_000)
```

```
    parser.add_argument("--n-envs", type=int, default=8)
    parser.add_argument("--n-steps", type=int, default=1024)
    parser.add_argument("--batch-size", type=int, default=64)
    parser.add_argument("--n-epochs", type=int, default=4)
    parser.add_argument("--gamma", type=float, default=0.999)
    parser.add_argument("--gae-lambda", type=float, default=0.98)
    parser.add_argument("--ent-coef", type=float, default=0.01)
    parser.add_argument("--vf-coef", type=float, default=0.5)
    parser.add_argument("--clip-range", type=float, default=0.2)
    parser.add_argument("--lr", type=float, default=3e-4)
    parser.add_argument("--lr-schedule", choices=["constant", "linear"], default="constant")
    parser.add_argument("--clip-schedule", choices=["constant", "linear"], default="constant")
    parser.add_argument("--n-stack", type=int, default=4, help="Frame stack (Atari only)")
    parser.add_argument("--atari", action="store_true",
                        help="Force Atari mode (CnnPolicy + atari wrappers). "
                             "Auto-detected from --env when it starts with ALE/.")
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--device", default="cuda:0")
    parser.add_argument("--out-dir", required=True)
    parser.add_argument("--eval-every", type=int, default=20_000)
    args = parser.parse_args()

    atari = args.atari or _is_atari(args.env)

    os.makedirs(args.out_dir, exist_ok=True)
    cfg = dict(vars(args))
    cfg["atari_detected"] = atari
    with open(os.path.join(args.out_dir, "config.json"), "w") as f:
        json.dump(cfg, f, indent=2)

    # Build training env
    if atari:
        _ensure_ale_registered()
        # SB3's make_atari_env applies the canonical preprocessing
        # (NoopReset, MaxAndSkip(4), EpisodicLife, FireReset, WarpFrame (84x84),
        # ClipReward). VecFrameStack adds the 4-frame stack.
        vec_env = make_atari_env(args.env, n_envs=args.n_envs, seed=args.seed)
        vec_env = VecFrameStack(vec_env, n_stack=args.n_stack)
        policy = "CnnPolicy"
    else:
        from stable_baselines3.common.monitor import Monitor

        def make_one(rank):
            def _thunk():
                env = gym.make(args.env)
                env = Monitor(env)
                env.reset(seed=args.seed + rank)
                return env
            return _thunk

```

```
        vec_env = DummyVecEnv([make_one(i) for i in range(args.n_envs)])
        policy = "MlpPolicy"

    lr = _linear_schedule(args.lr) if args.lr_schedule == "linear" else args.lr
    clip = _linear_schedule(args.clip_range) if args.clip_schedule == "linear" else args.clip_range

    model = PPO(
        policy,
        vec_env,
        n_steps=args.n_steps,
        batch_size=args.batch_size,
        n_epochs=args.n_epochs,
        gamma=args.gamma,
        gae_lambda=args.gae_lambda,
        ent_coef=args.ent_coef,
        vf_coef=args.vf_coef,
        clip_range=clip,
        learning_rate=lr,
        seed=args.seed,
        device=args.device,
        verbose=0,
    )

    cb = HistoryCallback(args.env, eval_every=args.eval_every, n_eval=10,
                         seed=args.seed, atari=atari)
    t0 = time.time()
    model.learn(total_timesteps=args.total_steps, callback=cb, progress_bar=False)
    elapsed = time.time() - t0

    # Final eval
    mean_r, std_r = cb._eval()
    cb.history["final_eval_mean"] = mean_r
    cb.history["final_eval_std"] = std_r
    cb.history["wall_time_sec"] = elapsed
    with open(os.path.join(args.out_dir, "history.json"), "w") as f:
        json.dump(cb.history, f)
    model.save(os.path.join(args.out_dir, "model.zip"))
    print(f"[PPO] DONE elapsed={elapsed/60:.1f}min final={mean_r:.2f}+/-{std_r:.2f}")


if __name__ == "__main__":
    main()
```

evaluate.py – DQN final 100-ep evaluation

```
"""Run final greedy evaluation on a trained model. Saves a JSON of per-episode
returns and prints summary statistics."""
import argparse
import json
import os

```

```python
import numpy as np
import torch
import gymnasium as gym

from dqn_agent import DQNAgent
from wrappers import make_atari


def make_env(env_id, seed):
    if env_id.startswith("ALE/"):
        return make_atari(env_id, seed=seed)
    return gym.make(env_id)


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--env", required=True)
    parser.add_argument("--variant", choices=["vanilla", "double", "dueling"], default="vanilla")
    parser.add_argument("--model", required=True)
    parser.add_argument("--n-episodes", type=int, default=100)
    parser.add_argument("--epsilon", type=float, default=0.01)
    parser.add_argument("--device", default="cuda:0")
    parser.add_argument("--seed", type=int, default=12345)
    parser.add_argument("--out", required=True)
    args = parser.parse_args()

    env = make_env(args.env, args.seed)
    agent = DQNAgent(env.observation_space, env.action_space.n, variant=args.variant, device=args.device)
    agent.load(args.model)

    returns = []
    lengths = []
    for ep in range(args.n_episodes):
        obs, _ = env.reset(seed=args.seed + ep)
        done = False
        total = 0.0
        steps = 0
        while not done:
            a = agent.act(obs, epsilon=args.epsilon)
            obs, r, term, trunc, _ = env.step(a)
            done = term or trunc
            total += r
            steps += 1
        returns.append(float(total))
        lengths.append(int(steps))

    summary = {
        "env": args.env,
        "variant": args.variant,
        "n_episodes": args.n_episodes,
        "mean": float(np.mean(returns)),
        "std": float(np.std(returns)),
        "min": float(np.min(returns)),
        "max": float(np.max(returns)),
        "median": float(np.median(returns)),
        "returns": returns,
```

```
        "lengths": lengths,
    }
    with open(args.out, "w") as f:
        json.dump(summary, f, indent=2)
    print(f"[{args.variant}] {args.env} | mean={summary['mean']:.2f} std={summary['std']:.2f} "
          f"min={summary['min']:.1f} max={summary['max']:.1f} (n={args.n_episodes})")


if __name__ == "__main__":
    main()
```

evaluate_ppo.py – PPO final evaluation

```
"""Greedy eval for a trained SB3 PPO model. Supports MLP envs (LunarLander)
and Atari envs (CnnPolicy + the standard atari wrappers + frame stack).
"""
import argparse
import json

import numpy as np
import gymnasium as gym
from stable_baselines3 import PPO
from stable_baselines3.common.env_util import make_atari_env
from stable_baselines3.common.vec_env import VecFrameStack


def _is_atari(env_id: str) -> bool:
    return env_id.startswith("ALE/") or "NoFrameskip" in env_id


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--env", default="LunarLander-v3")
    parser.add_argument("--model", required=True)
    parser.add_argument("--n-episodes", type=int, default=100)
    parser.add_argument("--seed", type=int, default=12345)
    parser.add_argument("--n-stack", type=int, default=4)
    parser.add_argument("--out", required=True)
    args = parser.parse_args()

    atari = _is_atari(args.env)
    model = PPO.load(args.model, device="cpu")

    returns, lengths = [], []
    if atari:
        import ale_py
        gym.register_envs(ale_py)
        # Single-env VecEnv that auto-resets after `done`. Build n_episodes
        # rollouts by reading the Monitor info dict, which carries the
        # episode return *before* any reward clipping (raw game score).
        env = make_atari_env(args.env, n_envs=1, seed=args.seed)
        env = VecFrameStack(env, n_stack=args.n_stack)
        obs = env.reset()
        steps = 0
```

```
        while len(returns) < args.n_episodes:
            a, _ = model.predict(obs, deterministic=True)
            obs, r, dones, infos = env.step(a)
            steps += 1
            if dones[0]:
                raw_return = float(infos[0]["episode"]["r"]) if "
                    episode" in infos[0] else float(r[0])
                returns.append(raw_return)
                lengths.append(int(infos[0]["episode"]["l"]) if "
                    episode" in infos[0] else steps)
                steps = 0
        env.close()
    else:
        env = gym.make(args.env)
        for ep in range(args.n_episodes):
            obs, _ = env.reset(seed=args.seed + ep)
            done = False
            total, steps = 0.0, 0
            while not done:
                a, _ = model.predict(obs, deterministic=True)
                obs, r, term, trunc, _ = env.step(a)
                done = term or trunc
                total += r
                steps += 1
            returns.append(float(total))
            lengths.append(int(steps))

    summary = {
        "env": args.env,
        "algo": "PPO",
        "n_episodes": args.n_episodes,
        "mean": float(np.mean(returns)),
        "std": float(np.std(returns)),
        "min": float(np.min(returns)),
        "max": float(np.max(returns)),
        "median": float(np.median(returns)),
        "returns": returns,
        "lengths": lengths,
    }
    with open(args.out, "w") as f:
        json.dump(summary, f, indent=2)
    print(f"[PPO] {args.env} | mean={summary['mean']:.2f} std={summary
        ['std']:.2f}")


if __name__ == "__main__":
    main()
```

evaluate_checkpoints.py – Per-checkpoint eval curves

```
"""Run greedy evaluation on every saved checkpoint (model_step*.pt) and
inject the results back into history.json under eval_step/eval_mean/
    eval_std.

This decouples evaluation from training -- if eval crashes (which we
    saw on
this setup) we don't lose training progress.
"""
```

```python
import argparse
import glob
import json
import os
import re

import numpy as np
import gymnasium as gym

from dqn_agent import DQNAgent
from wrappers import make_atari


def make_env(env_id, seed):
    if env_id.startswith("ALE/"):
        return make_atari(env_id, seed=seed)
    return gym.make(env_id)


def evaluate(agent, env_id, episodes, seed):
    env = make_env(env_id, seed)
    rets = []
    for ep in range(episodes):
        obs, _ = env.reset(seed=seed + ep)
        done = False
        total = 0.0
        while not done:
            a = agent.act(obs, epsilon=0.01)
            obs, r, term, trunc, _ = env.step(a)
            done = term or trunc
            total += r
        rets.append(float(total))
    env.close()
    return rets


def step_from_filename(p):
    m = re.search(r"model_step(\d+)\.pt$", p)
    return int(m.group(1)) if m else -1


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--env", required=True)
    parser.add_argument("--variant", choices=["vanilla", "double", "
        dueling"], required=True)
    parser.add_argument("--results-dir", required=True)
    parser.add_argument("--episodes", type=int, default=5)
    parser.add_argument("--device", default="cuda:0")
    parser.add_argument("--seed", type=int, default=100000)
    args = parser.parse_args()

    ckpts = sorted(glob.glob(os.path.join(args.results_dir, "model_step
        *.pt")), key=step_from_filename)
    if not ckpts:
        print("No model_step*.pt checkpoints found in", args.
            results_dir)
        return
```

```
62
63      # Build agent from any checkpoint to get obs_space; load history.
64      env = make_env(args.env, args.seed)
65      agent = DQNAgent(env.observation_space, env.action_space.n, variant
            =args.variant, device=args.device)
66      env.close()
67
68      hist_path = os.path.join(args.results_dir, "history.json")
69      with open(hist_path) as f:
70          history = json.load(f)
71
72      eval_step = list(history.get("eval_step", []))
73      eval_mean = list(history.get("eval_mean", []))
74      eval_std = list(history.get("eval_std", []))
75
76      done_steps = set(eval_step)
77      for ckpt in ckpts:
78          step = step_from_filename(ckpt)
79          if step in done_steps:
80              continue
81          agent.load(ckpt)
82          rets = evaluate(agent, args.env, args.episodes, args.seed)
83          m, s = float(np.mean(rets)), float(np.std(rets))
84          eval_step.append(step); eval_mean.append(m); eval_std.append(s)
85          print(f"step={step:>8d} mean={m:>7.2f} std={s:>5.2f} ({args.
                episodes} eps)")
86
87      # sort by step
88      order = np.argsort(eval_step)
89      history["eval_step"] = [int(eval_step[i]) for i in order]
90      history["eval_mean"] = [float(eval_mean[i]) for i in order]
91      history["eval_std"] = [float(eval_std[i]) for i in order]
92      with open(hist_path, "w") as f:
93          json.dump(history, f)
94      print(f"Updated {hist_path} with {len(history['eval_step'])} eval
            points")
95
96
97  if __name__ == "__main__":
98      main()
```

clean_history.py – De-duplicate crash-resumed history

```
1   """Post-process a history.json so the per-episode arrays are sorted by
        env
2   step and each step appears at most once. Necessary because crash+resume
        runs
3   duplicate (step, return) pairs spanning the lost progress."""
4   import argparse
5   import json
6   import os
7   import sys
8
9
10  def clean(history):
11      pairs = list(zip(history.get("step", []),
12                       history.get("ep_return", []),
13                       history.get("ep_len", [])))
```

```
    if not pairs:
        return history
    # Sort by step, then dedup keeping the LAST occurrence per step (later
    # entries reflect the most recent training pass through that env step).
    pairs.sort(key=lambda t: t[0])
    seen = {}
    for s, r, l in pairs:
        seen[s] = (r, l)
    out_steps = sorted(seen.keys())
    history["step"] = out_steps
    history["ep_return"] = [seen[s][0] for s in out_steps]
    history["ep_len"] = [seen[s][1] for s in out_steps]
    # Same for loss_step/loss/q_mean
    lpairs = list(zip(history.get("loss_step", []),
                      history.get("loss", []),
                      history.get("q_mean", [])))
    if lpairs:
        lpairs.sort(key=lambda t: t[0])
        lseen = {}
        for s, l, q in lpairs:
            lseen[s] = (l, q)
        out_l = sorted(lseen.keys())
        history["loss_step"] = out_l
        history["loss"] = [lseen[s][0] for s in out_l]
        history["q_mean"] = [lseen[s][1] for s in out_l]
    # Eval
    epairs = list(zip(history.get("eval_step", []),
                      history.get("eval_mean", []),
                      history.get("eval_std", [])))
    if epairs:
        epairs.sort(key=lambda t: t[0])
        eseen = {}
        for s, m, d in epairs:
            eseen[s] = (m, d)
        out_e = sorted(eseen.keys())
        history["eval_step"] = out_e
        history["eval_mean"] = [eseen[s][0] for s in out_e]
        history["eval_std"] = [eseen[s][1] for s in out_e]
    return history


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("paths", nargs="+", help="history.json files to clean (in-place)")
    args = parser.parse_args()
    for p in args.paths:
        if not os.path.exists(p):
            print(f"skip (missing): {p}")
            continue
        with open(p) as f:
            h = json.load(f)
        before = len(h.get("step", []))
        h2 = clean(h)
        after = len(h2.get("step", []))
        with open(p, "w") as f:
```

```
            json.dump(h2, f)
        print(f"{p}: {before} -> {after} episodes")


if __name__ == "__main__":
    main()
```

plot_results.py – Learning curves

```
"""Generate all plots and LaTeX tables for the report from training
    histories."""
import argparse
import json
import os

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

plt.rcParams.update({
    "font.size": 11,
    "axes.titlesize": 12,
    "axes.labelsize": 11,
    "legend.fontsize": 10,
    "xtick.labelsize": 10,
    "ytick.labelsize": 10,
    "figure.dpi": 130,
})


def load(path):
    with open(path) as f:
        return json.load(f)


def smooth(x, k=20):
    x = np.asarray(x, dtype=float)
    if len(x) < k:
        return x
    pad = np.full(k - 1, x[0])
    return np.convolve(np.concatenate([pad, x]), np.ones(k) / k, mode="
        valid")


def plot_learning_curves(runs, out_path, title, ylabel="Episode return"
    , x_scale=1.0):
    """runs: list of (label, history_dict, color).
    x_scale=1e-6 to display steps in millions, etc."""
    fig, ax = plt.subplots(figsize=(7.0, 4.2))
    for label, h, color in runs:
        steps = np.asarray(h["step"]) * x_scale
        rets = np.asarray(h["ep_return"])
        if len(rets) == 0:
            continue
        smoothed = smooth(rets, k=30)
        ax.plot(steps, smoothed, label=label, color=color, linewidth
            =1.6)
```

```
        # Light raw scatter
        ax.scatter(steps, rets, s=2, color=color, alpha=0.15)
    ax.set_xlabel("Environment steps" if x_scale == 1.0 else "Environment steps (millions)" if x_scale == 1e-6 else "Environment steps (thousands)")
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    ax.grid(True, alpha=0.3)
    ax.legend(loc="lower right", framealpha=0.9)
    fig.tight_layout()
    fig.savefig(out_path)
    plt.close(fig)
    print(f"Saved {out_path}")


def plot_eval_curves(runs, out_path, title, x_scale=1.0):
    fig, ax = plt.subplots(figsize=(7.0, 4.2))
    for label, h, color in runs:
        if "eval_step" not in h or not h["eval_step"]:
            continue
        s = np.asarray(h["eval_step"]) * x_scale
        m = np.asarray(h["eval_mean"])
        d = np.asarray(h["eval_std"])
        ax.plot(s, m, label=label, color=color, marker="o", markersize=4, linewidth=1.6)
        ax.fill_between(s, m - d, m + d, color=color, alpha=0.15)
    ax.set_xlabel("Environment steps" if x_scale == 1.0 else "Environment steps (millions)" if x_scale == 1e-6 else "Environment steps (thousands)")
    ax.set_ylabel("Greedy eval mean return")
    ax.set_title(title)
    ax.grid(True, alpha=0.3)
    ax.legend(loc="lower right", framealpha=0.9)
    fig.tight_layout()
    fig.savefig(out_path)
    plt.close(fig)
    print(f"Saved {out_path}")


def plot_q_mean(runs, out_path, title):
    fig, ax = plt.subplots(figsize=(7.0, 4.2))
    for label, h, color in runs:
        if "loss_step" not in h or not h["loss_step"]:
            continue
        s = np.asarray(h["loss_step"]) * 1e-6
        q = np.asarray(h["q_mean"])
        ax.plot(s, smooth(q, k=50), label=label, color=color, linewidth=1.4)
    ax.set_xlabel("Environment steps (millions)")
    ax.set_ylabel("Mean predicted Q (batch avg)")
    ax.set_title(title)
    ax.grid(True, alpha=0.3)
    ax.legend(loc="upper left", framealpha=0.9)
    fig.tight_layout()
    fig.savefig(out_path)
    plt.close(fig)
    print(f"Saved {out_path}")

```

```python

def bar_final(rows, out_path, title, ylabel="Mean return (100 eps)"):
    """rows: list of (label, mean, std, color)."""
    fig, ax = plt.subplots(figsize=(6.0, 3.8))
    xs = np.arange(len(rows))
    means = [r[1] for r in rows]
    stds = [r[2] for r in rows]
    colors = [r[3] for r in rows]
    ax.bar(xs, means, yerr=stds, color=colors, capsize=6, alpha=0.85,
        edgecolor="black", linewidth=0.7)
    ax.set_xticks(xs)
    ax.set_xticklabels([r[0] for r in rows], rotation=10)
    ax.axhline(0, color="black", linewidth=0.6)
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    ax.grid(True, axis="y", alpha=0.3)
    fig.tight_layout()
    fig.savefig(out_path)
    plt.close(fig)
    print(f"Saved {out_path}")


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--results-dir", default="/mnt/nvme0n1/
        ia313553058/Others/AI_3/results")
    parser.add_argument("--out-dir", default="/mnt/nvme0n1/ia313553058/
        Others/AI_3/report/figures")
    args = parser.parse_args()

    os.makedirs(args.out_dir, exist_ok=True)
    R = args.results_dir

    # ------- Pong (DQN variants only) -------
    try:
        runs = []
        for label, sub, color in [
            ("Vanilla DQN", "pong_vanilla", "tab:blue"),
            ("Double DQN", "pong_double", "tab:orange"),
            ("Dueling Double DQN", "pong_dueling", "tab:green"),
        ]:
            p = f"{R}/{sub}/history.json"
            if os.path.exists(p):
                runs.append((label, load(p), color))
        if runs:
            plot_learning_curves(runs, f"{args.out_dir}/pong_learning.
                pdf",
                                 "Pong: training-time episode return
                                    (30-ep MA)",
                                 x_scale=1e-6)
            plot_eval_curves(runs, f"{args.out_dir}/pong_eval.pdf",
                             "Pong: greedy evaluation (5 eps/checkpoint
                                )",
                             x_scale=1e-6)
            plot_q_mean(runs, f"{args.out_dir}/pong_qmean.pdf",
                        "Pong: mean predicted Q-value over training")
    except FileNotFoundError as e:
        print(f"Skipping Pong plots (missing history): {e}")
```

```
150
151     # ------- Pong: DQN vs PPO comparison at matched 1M budget -------
152     try:
153         runs = []
154         for label, sub, color in [
155             ("Vanilla DQN", "pong_vanilla", "tab:blue"),
156             ("Double DQN", "pong_double", "tab:orange"),
157             ("Dueling Double DQN", "pong_dueling", "tab:green"),
158             ("PPO (CnnPolicy)", "pong_ppo_s42", "tab:red"),
159         ]:
160             p = f"{R}/{sub}/history.json"
161             if os.path.exists(p):
162                 runs.append((label, load(p), color))
163         if any(r[0].startswith("PPO") for r in runs):
164             plot_learning_curves(
165                 runs, f"{args.out_dir}/pong_dqn_vs_ppo.pdf",
166                 "Pong: DQN variants vs PPO (training-time return, 30-ep
                        MA)",
167                 x_scale=1e-6,
168             )
169     except FileNotFoundError:
170         pass
171
172     # ------- LunarLander DQN vs PPO -------
173     try:
174         ll_runs = []
175         for label, sub, color in [
176             ("DQN (Double)", "lander_double", "tab:orange"),
177             ("DQN (Dueling)", "lander_dueling", "tab:green"),
178             ("PPO", "lander_ppo", "tab:red"),
179         ]:
180             p = f"{R}/{sub}/history.json"
181             if os.path.exists(p):
182                 ll_runs.append((label, load(p), color))
183         if ll_runs:
184             plot_learning_curves(ll_runs, f"{args.out_dir}/
                    lander_learning.pdf",
185                                  "LunarLander: training-time episode
                                     return (30-ep MA)",
186                                  x_scale=1e-3)
187             plot_eval_curves(ll_runs, f"{args.out_dir}/lander_eval.pdf"
                    ,
188                              "LunarLander: greedy evaluation curve",
189                              x_scale=1e-3)
190     except FileNotFoundError as e:
191         print(f"Skipping LunarLander plots: {e}")
192
193     # ------- LunarLander hyperparameter ablation (learning rate)
            -------
194     try:
195         rows = []
196         for label, sub, color in [
197             (r"lr = $1\times 10^{-3}$", "lander_lr1e-3", "tab:purple"),
198             (r"lr = $3\times 10^{-4}$ (default)", "lander_double", "tab
                    :gray"),
199             (r"lr = $1\times 10^{-4}$", "lander_lr1e-4", "tab:brown"),
200         ]:
201             p = f"{R}/{sub}/history.json"
```

```
            if os.path.exists(p):
                rows.append((label, load(p), color))
        if rows:
            plot_learning_curves(rows, f"{args.out_dir}/lander_lr_ablation.pdf",
                                 "LunarLander Double DQN: learning-rate sensitivity",
                                 x_scale=1e-3)
    except FileNotFoundError:
        pass


if __name__ == "__main__":
    main()
```

make_tables.py – Result tables

```
"""Generate LaTeX tables for the report from training/evaluation JSONs."""
import argparse
import json
import os

import numpy as np


def load(p):
    if not os.path.exists(p):
        return None
    with open(p) as f:
        return json.load(f)


def pong_final_table(results_dir, out_path):
    """Compare final eval scores across DQN variants on Pong."""
    rows = []
    for label, sub in [
        ("Vanilla DQN", "pong_vanilla"),
        ("Double DQN", "pong_double"),
        ("Dueling Double DQN", "pong_dueling"),
    ]:
        h = load(os.path.join(results_dir, sub, "history.json"))
        e = load(os.path.join(results_dir, sub, "eval100.json"))
        if h is None and e is None:
            continue
        # Use 100-ep eval if available, else final_eval from history
        if e is not None:
            mean, std = e["mean"], e["std"]
            minv, maxv, n = e["min"], e["max"], e["n_episodes"]
        else:
            mean, std = h.get("final_eval_mean", float("nan")), h.get("final_eval_std", float("nan"))
            minv = maxv = float("nan"); n = 20
        # Compute training-time best from 100-ep moving avg
        rets = np.asarray(h["ep_return"]) if h else np.array([])
        if len(rets) >= 100:
            ma = np.convolve(rets, np.ones(100) / 100, mode="valid")
            best_train = float(np.max(ma))
```

```
        else:
            best_train = float("nan")
        rows.append((label, mean, std, minv, maxv, n, best_train))

    s = []
    s.append(r"\begin{table}[h]\centering\small")
    s.append(r"\caption{Pong: final greedy evaluation (100 episodes) and best 100-episode "
             r"training-time average across DQN variants. All variants share identical "
             r"hyperparameters, replay buffer, and seed.}")
    s.append(r"\label{tab:pong_final}")
    s.append(r"\begin{tabular}{lrrrrrr}")
    s.append(r"\toprule")
    s.append(r"Variant & Mean & Std & Min & Max & $n$ & Best train (100-ep MA) \\")
    s.append(r"\midrule")
    for label, mean, std, mn, mx, n, bt in rows:
        s.append(f"{label} & {mean:.2f} & {std:.2f} & {mn:.1f} & {mx:.1f} & {n} & {bt:.2f} \\\\")
    s.append(r"\bottomrule")
    s.append(r"\end{tabular}")
    s.append(r"\end{table}")
    with open(out_path, "w") as f:
        f.write("\n".join(s) + "\n")
    print(f"Wrote {out_path}")


def lander_final_table(results_dir, out_path):
    """Compare LunarLander DQN variants and PPO."""
    rows = []
    for label, sub, kind in [
        ("DQN (Double)", "lander_double", "dqn"),
        ("DQN (Dueling Double)", "lander_dueling", "dqn"),
        ("PPO", "lander_ppo", "ppo"),
    ]:
        h = load(os.path.join(results_dir, sub, "history.json"))
        e = load(os.path.join(results_dir, sub, "eval100.json"))
        if e is None:
            e = load(os.path.join(results_dir, sub, "eval100_partial.json"))
        if h is None and e is None:
            continue
        if e is not None:
            mean, std = e["mean"], e["std"]
            mn, mx, n = e["min"], e["max"], e["n_episodes"]
        else:
            # Fallback to training-loop's final small eval (10 eps for PPO).
            mean = h.get("final_eval_mean", float("nan"))
            std = h.get("final_eval_std", float("nan"))
            mn = mx = float("nan"); n = 10
        rets = np.asarray(h["ep_return"]) if h else np.array([])
        if len(rets) >= 100:
            ma = np.convolve(rets, np.ones(100) / 100, mode="valid")
            best_train = float(np.max(ma))
        else:
            best_train = float("nan")
```

```
        wt = h.get("wall_time_sec", float("nan")) if h else float("nan")
        rows.append((label, mean, std, mn, mx, n, best_train, wt))

    s = []
    s.append(r"\begin{table}[h]\centering\small")
    s.append(r"\caption{LunarLander: final greedy/deterministic evaluation. "
             r"DQN runs are 100 greedy ($\varepsilon=0.01$) episodes from a 500\,K-step model; "
             r"PPO is 30 deterministic episodes from a 250\,K-timestep model (longer SB3 runs "
             r"crashed; see Discussion). Solved threshold is mean return $\geq 200$ (bold).}")
    s.append(r"\label{tab:lander_final}")
    s.append(r"\begin{tabular}{lrrrrrr}")
    s.append(r"\toprule")
    s.append(r"Algorithm & Mean & Std & Min & Max & $n$ & Best train (100-ep MA) \\")
    s.append(r"\midrule")
    for label, mean, std, mn, mx, n, bt, _ in rows:
        bold = mean >= 200
        if bold:
            line = f"\\textbf{{{label}}} & \\textbf{{{mean:.2f}}} & {std:.2f} & {mn:.1f} & {mx:.1f} & {n} & {bt:.2f} \\\\"
        else:
            line = f"{label} & {mean:.2f} & {std:.2f} & {mn:.1f} & {mx:.1f} & {n} & {bt:.2f} \\\\"
        s.append(line)
    s.append(r"\bottomrule")
    s.append(r"\end{tabular}")
    s.append(r"\end{table}")
    with open(out_path, "w") as f:
        f.write("\n".join(s) + "\n")
    print(f"Wrote {out_path}")


def lr_ablation_table(results_dir, out_path):
    rows = []
    for label, sub in [
        ("$1\\times 10^{-3}$", "lander_lr1e-3"),
        ("$3\\times 10^{-4}$ (default)", "lander_lr3e-4"),
        ("$1\\times 10^{-4}$", "lander_lr1e-4"),
    ]:
        h = load(os.path.join(results_dir, sub, "history.json"))
        e = load(os.path.join(results_dir, sub, "eval100.json"))
        if h is None and e is None:
            continue
        if e is not None:
            mean, std = e["mean"], e["std"]
            n = e["n_episodes"]
        else:
            mean = h.get("final_eval_mean", float("nan"))
            std = h.get("final_eval_std", float("nan"))
            n = 10
        rets = np.asarray(h["ep_return"]) if h else np.array([])
        if len(rets) >= 100:
            ma = np.convolve(rets, np.ones(100) / 100, mode="valid")
```

```
            best_train = float(np.max(ma))
        else:
            best_train = float("nan")
        rows.append((label, mean, std, best_train, n))

    s = []
    s.append(r"\begin{table}[h]\centering\small")
    s.append(r"\caption{LunarLander DQN (Double): learning-rate sensitivity. All other "
             r"hyperparameters held fixed; same seed.}")
    s.append(r"\label{tab:lr_ablation}")
    s.append(r"\begin{tabular}{lrrrr}")
    s.append(r"\toprule")
    s.append(r"Learning rate & Final eval mean & Std & Best train (100-ep MA) & $n$ \\")
    s.append(r"\midrule")
    for label, mean, std, bt, n in rows:
        s.append(f"{label} & {mean:.2f} & {std:.2f} & {bt:.2f} & {n} \\\\")
    s.append(r"\bottomrule")
    s.append(r"\end{tabular}")
    s.append(r"\end{table}")
    with open(out_path, "w") as f:
        f.write("\n".join(s) + "\n")
    print(f"Wrote {out_path}")


def hyperparam_table(out_path):
    """Static hyperparameter table."""
    s = []
    s.append(r"\begin{table}[h]\centering\small")
    s.append(r"\caption{Key hyperparameters. DQN settings follow \citet{mnih2015human} "
             r"with a slightly shorter exploration ramp and replay buffer to fit our compute budget. "
             r"PPO settings follow the SB3 LunarLander tuning.}")
    s.append(r"\label{tab:hyperparams}")
    s.append(r"\begin{tabular}{lll}")
    s.append(r"\toprule")
    s.append(r"Hyperparameter & Pong DQN family & LunarLander DQN \\")
    s.append(r"\midrule")
    s.append(r"Total environment steps & 2{,}000{,}000 & 500{,}000 \\")
    s.append(r"Optimizer & Adam ($\beta_1=0.9,\beta_2=0.999$) & Adam \\")
    s.append(r"Learning rate & $1\!\times\!10^{-4}$ & $3\!\times\!10^{-4}$ \\")
    s.append(r"Discount $\gamma$ & 0.99 & 0.99 \\")
    s.append(r"Replay buffer size & 100{,}000 & 100{,}000 \\")
    s.append(r"Batch size & 32 & 64 \\")
    s.append(r"Learning starts at step & 50{,}000 & 5{,}000 \\")
    s.append(r"Train freq (env steps) & 4 & 4 \\")
    s.append(r"Target net sync (steps) & 1{,}000 & 1{,}000 \\")
    s.append(r"Exploration $\varepsilon$ start $\to$ end & $1.0\to 0.01$ & $1.0\to 0.05$ \\")
    s.append(r"Exploration linear-decay fraction & $0.10$ & $0.10$ \\")
    s.append(r"Loss & Huber & Huber \\")
    s.append(r"Gradient clipping & 10.0 & 10.0 \\")
    s.append(r"\midrule")
```

```
    s.append(r"\textbf{PPO (LunarLander)} & \multicolumn{2}{l}{}\\")
    s.append(r"Parallel envs & \multicolumn{2}{l}{8} \\")
    s.append(r"Rollout length & \multicolumn{2}{l}{1024 steps / env} \\")
    s.append(r"Mini-batch size & \multicolumn{2}{l}{64} \\")
    s.append(r"Update epochs & \multicolumn{2}{l}{4} \\")
    s.append(r"GAE $\lambda$ & \multicolumn{2}{l}{0.98} \\")
    s.append(r"$\gamma$ & \multicolumn{2}{l}{0.999} \\")
    s.append(r"Entropy coefficient & \multicolumn{2}{l}{0.01} \\")
    s.append(r"Clip $\epsilon$ & \multicolumn{2}{l}{0.2 (SB3 default)} \\")
    s.append(r"Learning rate & \multicolumn{2}{l}{$3\!\times\!10^{-4}$} \\")
    s.append(r"\bottomrule")
    s.append(r"\end{tabular}")
    s.append(r"\end{table}")
    with open(out_path, "w") as f:
        f.write("\n".join(s) + "\n")
    print(f"Wrote {out_path}")


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--results-dir", default="/mnt/nvme0n1/ia313553058/Others/AI_3/results")
    parser.add_argument("--out-dir", default="/mnt/nvme0n1/ia313553058/Others/AI_3/report/tables")
    args = parser.parse_args()
    os.makedirs(args.out_dir, exist_ok=True)
    hyperparam_table(os.path.join(args.out_dir, "hyperparams.tex"))
    pong_final_table(args.results_dir, os.path.join(args.out_dir, "pong_final.tex"))
    lander_final_table(args.results_dir, os.path.join(args.out_dir, "lander_final.tex"))
    lr_ablation_table(args.results_dir, os.path.join(args.out_dir, "lr_ablation.tex"))
```